LectureLens Windows Edition — Master Deployment Guide

For: D&R AI Solutions — production deployment of Windows Edition end-to-end **Estimated total time:** 2 hours (first time) · 30 min (subsequent rooms) **Components:** Backend Windows endpoints + Admin portal Windows section + Windows Mini PC app

---

Big picture — what gets deployed where

Component	Target	Deploy method	Time
Backend Windows endpoints	Existing Railway deployment	`git push` to main (auto-deploys)	5 min
Admin portal Windows section	Existing Vercel deployment	`vercel deploy --prod`	5 min
Windows app on Mini PC	Per-classroom Mini PC	MSIX/manual install + service register	30 min/PC

The Windows Edition is **fully isolated** from the Android Edition — none of the existing Android code, schemas, routes, or admin portal pages are modified. Both editions coexist on the same backend instance.

Phase A — Backend deployment (Railway)

Step A.1 — Verify the new Windows files are in place

The following NEW files exist in the backend (already created — do not modify Android files):

```
backend/
├── models/windows/
│   ├── WindowsDevice.js
│   ├── WindowsRecording.js
│   ├── WindowsLicense.js
│   └── WindowsDeviceCommand.js
├── controllers/windows/
│   ├── deviceController.js
│   ├── recordingController.js
│   └── licenseController.js
├── routes/windows/
│   └── index.js
├── middleware/
│   └── windowsDeviceAuth.js
└── index.js  ← MODIFIED: added one line registering /api/windows route
```

Only one existing file modified: `backend/index.js` — added a single line to register the Windows route group. Verify:

```
grep "/api/windows" backend/index.js
# Should output:  app.use("/api/windows", require("./routes/windows"));
```

Step A.2 — Commit and push

```
cd lecture-capture-system
git add backend/models/windows backend/controllers/windows backend/routes/windows backend/m
git commit -m "Add Windows Edition: routes, models, license management"
git push origin main
```

Railway auto-deploys from `main` branch within ~2 minutes.

Step A.3 — Verify backend deployment

```
# Wait ~2 min for Railway to redeploy, then:
TOKEN=$(curl -sS -X POST "https://lecturelens-api.draisol.com/api/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email":"adminportal@draisol.com","password":"admin@123"}' | python3 -c "import json

# Test new Windows devices endpoint (should return empty list, not 404)
curl -sS "https://lecturelens-api.draisol.com/api/windows/devices" \
  -H "Authorization: Bearer $TOKEN"
# Expected: {"devices":[]}
```

If you see `Cannot GET /api/windows/devices`, the deploy hasn't completed. Wait another minute and retry.

Phase B — Admin portal deployment (Vercel)

Step B.1 — Verify new Windows files in admin portal

```
admin-portal/src/
├── pages/windows/
│   ├── WindowsDashboard.jsx
│   ├── WindowsDevices.jsx
│   ├── WindowsRecordings.jsx
│   ├── WindowsBooking.jsx
│   ├── WindowsLicenses.jsx
│   └── windows.css
├── services/
│   └── windowsApi.js   ← NEW (separate from existing api.js)
├── App.jsx             ← MODIFIED: added 5 routes under /windows/*
└── components/Layout.jsx  ← MODIFIED: added "Windows" sidebar group
```

Only two existing files modified (additive, no Android code touched): `App.jsx` and `Layout.jsx`.

Step B.2 — Build & deploy

```
cd admin-portal
npm install   # in case any new dependencies (none added in this release, but safe)
vercel deploy --prod --yes
```

Step B.3 — Verify in browser

1. Open `https://lecturelens-admin.draisol.com` (or your production URL)
2. Login with admin credentials
3. Sidebar should show TWO groups: - **ANDROID** (existing items — unchanged) - **WINDOWS** with NEW badge (5 new items)
4. Click **Windows** → **Dashboard** — should load without errors
5. Click **Windows** → **Devices** — empty list (no devices registered yet)
6. Click **Windows** → **Licenses** — empty list with "Issue New License" button
7. Click **Windows** → **Booking** — empty state ("No Windows devices registered yet")

If pages load: deployment successful.

Step B.4 — Issue a test license

While in admin portal: 1. Navigate to **Windows** → **Licenses** 2. Click **+ Issue New License** 3. Fill form: - Tier: **Professional** - Customer Name: `Test Pilot` - Customer Email: your email - Expires: any future date (e.g., 1 year out) 4. Click **Issue License** 5. **COPY the license key** that appears (e.g., `WIN-7A3K-9X2P-NLM4-Q8RY`) — needed for next phase

---

Phase C — Windows Mini PC deployment

For full step-by-step instructions, see `lecturelens-windows-recorder/SETUP-STEP-BY-STEP.md`. Summary here:

Step C.1 — Hardware checklist (per Mini PC)

- [ ] Windows Mini PC (Intel NUC i5-13500T or equivalent recommended)
- [ ] Display connected via HDMI
- [ ] Lumens VC-TR1 camera on campus Ethernet (RTSP reachable)
- [ ] Sennheiser TCC2 → Extron DMP Plus → Mini PC USB
- [ ] Ethernet to campus switch (don't use WiFi)
- [ ] UPS recommended

Step C.2 — Build Windows app on a dev machine

```
# On a Windows 10/11 machine with .NET 8 SDK installed
cd lecturelens-windows-recorder
.\scripts\build.ps1
```

Output appears in `src\LectureLens.WindowsRecorder\bin\Release\net8.0-windows\publish\`.

Step C.3 — Install OBS Studio on the target Mini PC

1. Download OBS Studio 30+ from https://obsproject.com
2. Install with defaults (path: `C:\Program Files\obs-studio\`)
3. Launch OBS once → **Tools** → **WebSocket Server Settings** - Check "Enable WebSocket server" - Port: `4455` - Click "Show Connect Info" → **copy the Server Password**
4. Configure scene with sources: - **Media Source** → name "Lumens Camera" → Input: `rtsp://<your-camera-ip>:554/live/main` - **Audio Input Capture** → name "DMP Plus" → Device: select Extron DMP Plus - (Optional) **Display Capture** → for screen content

Step C.4 — Copy build output to Mini PC

Either via USB drive, network share, or download. Drop it in `C:\Temp\LectureLens-Build\`.

Step C.5 — Install service (as Administrator on the Mini PC)

```
# Open PowerShell as Administrator
cd C:\path\to\lecturelens-windows-recorder
.\scripts\install-service.ps1 -BinarySource "C:\Temp\LectureLens-Build"
```

This: - Copies binaries to `C:\Program Files\LectureLens\` - Creates `C:\LectureLens\Recordings\` + `C:\LectureLens\logs\` - Registers Windows Service with auto-start + restart-on-failure - Starts the service

Step C.6 — Configure the service

```
notepad "C:\Program Files\LectureLens\appsettings.json"
```

Edit these fields:

```
{
  "LectureLens": {
    "DeviceId": "win_<generate-via-PowerShell:-[guid]::NewGuid()>",
    "AuthToken": "<obtained-from-/api/windows/devices/register-or-leave-blank-for-auto>",
    "LicenseKey": "WIN-7A3K-9X2P-NLM4-Q8RY",   ← from Phase B.4
    "RoomNumber": "101",                          ← this room's number
    "DeviceName": "Smart PC - Room 101",

    "BackendUrl": "https://lecturelens-api.draisol.com",
    "ObsWebSocketPassword": "<password-from-OBS-Phase-C.3>",

    "AzureBlobConnectionString": "DefaultEndpointsProtocol=https;AccountName=...",  ← from
    "CameraSource": "rtsp://<your-camera-ip>:554/live/main"
  }
}
```

Save & close.

Step C.7 — Register the device with the backend

If you didn't pre-generate `DeviceId` and `AuthToken`, the service can self-register on first heartbeat. Or do it manually via curl:

```
$body = @{
  name = "Smart PC - Room 101"
  roomNumber = "101"
  hardwareModel = "Intel NUC 13 Pro"
  osVersion = "Windows 11 IoT Enterprise LTSC"
} | ConvertTo-Json

$resp = Invoke-RestMethod -Uri "https://lecturelens-api.draisol.com/api/windows/devices/reg
  -Method Post -Body $body -ContentType "application/json"
$resp | ConvertTo-Json
# Copy "deviceId" and "authToken" into appsettings.json
```

Step C.8 — Restart service to apply config

```
Restart-Service LectureLensRecorder
```

Step C.9 — Verify the service is running

```
Get-Service LectureLensRecorder
# Status should be "Running"

# Watch live logs
Get-Content C:\LectureLens\logs\lecturelens-*.log -Tail 50 -Wait
```

You should see:

```
=== LectureLens Windows Recorder starting ===
Worker starting up...
Backend URL: https://lecturelens-api.draisol.com
License manager initializing...
OBS WebSocket connected
Connected to OBS 30.0.0
Heartbeat OK - schedule:0, commands:0
```

Step C.10 — Confirm device appears in admin portal

1. Open `https://lecturelens-admin.draisol.com` in browser
2. Navigate to **Windows** → **Devices**
3. The new Mini PC should appear as ONLINE with health metrics
4. License badge should show **professional** (or whatever tier you issued)

Phase D — End-to-end test

Step D.1 — Schedule a test class

1. In admin portal: **Windows** → **Booking**
2. Select your Windows-served room
3. Title: “Test Recording”
4. Date: today
5. Start time: **2 minutes from now** (give the schedule poller time)
6. End time: **5 minutes from now** (3-minute test)
7. Click **Schedule Class**

Step D.2 — Watch the recording happen

In the Mini PC PowerShell window:

```
Get-Content C:\LectureLens\logs\lecturelens-*.log -Tail 50 -Wait
```

When the class start time arrives, you should see:

```
Schedule transition: stopping <none> to start <new-class-id>
Starting recording for class <id>: Test Recording
OBS recording started
```

After 3 minutes:

```
Schedule end reached for <id>, stopping
OBS recording stopped → C:\LectureLens\Recordings\<id>\<filename>.mp4
Uploaded chunk <filename> → https://...azure.../full.mp4
```

Step D.3 — Verify in admin portal

1. **Windows** → **Recordings** — your test recording should appear
2. Status: `recording` → `uploading` → `completed`
3. Click **Play** to playback the MP4

If the MP4 plays with proper video + audio: **end-to-end working.** ✅

Phase E — Per-room rollout

Repeat **Phase C** for each additional Mini PC. Each room needs: - Its own license key (issue from admin portal) - Its own `DeviceId` (auto-generated on register) - Its own `RoomNumber` in config - Its own Lumens camera IP / RTSP URL in config

The first PC takes ~30 min. Subsequent ones go faster (~15 min) once you have the build artifact and a checklist.

Architecture isolation guarantees

This deployment introduces ZERO changes to Android Edition behavior:

Android component	Status
Existing Android APK	Untouched
`/api/classroom-recording/*` endpoints	Untouched
`lcs_classroomdevices`, `lcs_recordings` collections	Untouched
Admin portal Android pages: Devices, Recordings, Booking, App Update, Licenses	Untouched
Existing LiveKit pipeline + Egress	Untouched
Android OTA mechanism	Untouched
Class scheduling API (`/api/classes`)	Shared (platform-agnostic)

If Android stops working after this deploy: it's a coincidence, not caused by Windows Edition. Roll back the **last commit** to confirm — if Android still broken, root cause is elsewhere.

Troubleshooting

Backend returns `Cannot GET /api/windows/devices`

- Railway hasn't finished deploying. Wait 2 more minutes, retry.
- Check Railway logs for build errors.
- Confirm `app.use("/api/windows", require("./routes/windows"));` exists in `backend/index.js` and was committed.

Admin portal sidebar missing Windows section

- Hard reload (Cmd+Shift+R / Ctrl+Shift+F5) — Vercel may serve cached JS.
- Confirm `vercel deploy --prod` completed. Check the deployment URL in Vercel dashboard.
- Open browser console — any module-loading errors?

Windows service won't start

- Event Viewer → Windows Logs → Application → filter `LectureLensRecorder`
- Common fixes:
- `appsettings.json` invalid JSON → fix syntax
- Wrong path to `obs64.exe` → update `ObsExecutablePath`
- License key wrong format → must be `WIN-XXXX-XXXX-XXXX-XXXX`

"Could not connect to OBS WebSocket"

- Is OBS running? Try launch manually
- WebSocket enabled? **OBS → Tools → WebSocket Server Settings**
- Password matches between OBS and `appsettings.json`?
- Port 4455 not blocked: `New-NetFirewallRule -DisplayName "OBS WebSocket" -Direction Inbound -Protocol TCP -LocalPort 4455 -Action Allow`

Recording in admin portal but file is silent

- Check OBS Audio Mixer — is "DMP Plus" source level moving when you talk?
- If not, in OBS edit source properties → reselect device

License shows "INVALID"

- Check license key typed correctly (no spaces)
- Backend `/api/windows/licenses/<key>/validate` should return `{valid: true}`
- License might be already bound to a different device (one license = one device)

Pricing reference

Tier	INR / year	USD / year	Best for
Starter	₹20,000	$275	Limited use (<4hrs/day), 720p, no live-watch
Professional	₹45,000	$600	Standard classrooms, 1080p, live-watch, 12hrs/day
Enterprise	₹75,000	$1,000	Premium installations, 4K, multi-camera, unlimited

Volume discounts and multi-year prepay available — see the engineering doc.

Service management cheat sheet

```
# Start / Stop / Restart
Start-Service LectureLensRecorder
Stop-Service LectureLensRecorder
Restart-Service LectureLensRecorder

# Status
Get-Service LectureLensRecorder

# View logs (real-time)
Get-Content C:\LectureLens\logs\lecturelens-*.log -Tail 100 -Wait

# View recordings folder
explorer C:\LectureLens\Recordings

# Uninstall (preserves recordings + logs)
.\lecturelens-windows-recorder\scripts\uninstall-service.ps1
```

Final acceptance checklist

After full deployment, verify:

- [ ] Backend `/api/windows/devices` returns `{devices: []}` (or list with registered devices)
- [ ] Backend `/api/windows/licenses` returns `[]` or list with issued licenses
- [ ] Admin portal sidebar shows ANDROID + WINDOWS sections
- [ ] Admin portal `/windows/dashboard` page loads without errors
- [ ] Admin portal `/windows/licenses` can issue + revoke licenses
- [ ] Admin portal `/windows/devices` shows registered Mini PC(s)
- [ ] Admin portal `/windows/booking` lets you schedule a class
- [ ] Mini PC service auto-starts on boot
- [ ] Mini PC heartbeats appear in admin portal within 30 sec
- [ ] Test class records local file
- [ ] Test class chunk uploads to Azure Blob
- [ ] Test class appears in `/windows/recordings` with playable URL
- [ ] Android section in admin portal still works as before

Deployment complete. For support, contact D&R AI Solutions Engineering.

Document version: 1.0 — May 2026